

C#.NET



Multithreading

Part-4

Mr. Bangar Raju

 **NARESHO**
technologies

Understanding the Join Method in C# Multithreading

By Albert Harmon Published April 8, 2017 ⌚ 3 min read Technology.



TLDR This blog post explores the Join method in C# multithreading, explaining its purpose, how to implement it, and the differences between using it with and without a timeout. It emphasizes the importance of ensuring that the main

thread waits for all child threads to complete before exiting.

In this blog post, we will delve into the Join method in C# multithreading, a crucial aspect for managing thread execution. This is part four of a series on multithreading in C#, where we previously discussed the ThreadStart delegate and how to pass methods as parameters to threads.

What is the Join Method?

The Join method is a member of the Thread class in C#. It is used to make the calling thread wait until the thread on which Join is called has completed its execution. This is particularly important when you want to ensure that the main thread does not exit before all child threads have finished their tasks.

Implementing the Join Method

To illustrate the use of the Join method, let's consider a simple program. We will define a method and create multiple threads to execute it. Here's a basic outline of the code:

```
static void TestMethod(int i)
{
    for (int j = 0; j <= 25; j++)
```

```
{
    Console.WriteLine("Test method " + i + " executing: ")
}

static void Main()
{
    Console.WriteLine("Main thread started");

    Thread t1 = new Thread(() => TestMethod(1));
    Thread t2 = new Thread(() => TestMethod(2));
    Thread t3 = new Thread(() => TestMethod(3));

    t1.Start();
    t2.Start();
    t3.Start();

    Console.WriteLine("Main thread exiting");
}
```

Observing Thread Execution

When you run this code, you might notice that the main thread can exit before the child threads complete their execution. This is because the main thread does not wait for the child threads to finish. To prevent this, we can use the Join method:

```
t1.Join();  
t2.Join();  
t3.Join();
```

By adding these lines, we ensure that the main thread will wait for each child thread to finish before it exits. This guarantees that the output will reflect the completion of all threads before the main thread concludes.

Example Output

When the Join method is implemented, the output will show that the main thread exits last, confirming that it waited for all child threads to complete:

```
Main thread started  
Test method 1 executing: 0  
...  
Test method 3 executing: 25  
Main thread exiting
```

Using Join with Timeout

The Join method also has an overloaded version that accepts a timeout parameter. This allows the main thread to wait for a specified amount

of time for a child thread to complete. If the child thread does not finish within the timeout period, the main thread will continue executing.

For example:

```
t1.Join(3000); // Wait for 3 seconds
```

In this case, if `t1` does not complete within 3 seconds, the main thread will exit regardless of the state of `t1`. This can lead to different outputs depending on the timing of thread execution.

Example with Timeout

If `t1` is designed to sleep for 5 seconds, and we set a timeout of 3 seconds, the output will indicate that the main thread exited first:

```
Main thread started  
Test method 1 executing: 0  
...  
Main thread exited
```

This demonstrates the importance of understanding how the `Join` method works, both with and without a timeout, to manage thread execution effectively.

Conclusion

The Join method is an essential tool in C# multithreading that allows the main thread to wait for child threads to complete their execution. By using Join, developers can ensure that all threads finish their tasks before the main thread exits, leading to more predictable and reliable program behavior. Understanding the nuances of Join, including its timeout feature, can significantly enhance your multithreading capabilities in C#.